



Mi App del Clima

Todo el proyecto explicado como si tuvieras 5 años

Práctica 1 · React + Vite + OpenWeatherMap

1. ¿Qué es esta app? 🌍

Es una página web que te dice **qué clima hace** en cualquier ciudad del mundo. Escribes "Roma" y te muestra: la temperatura, si está nublado o soleado, cuánto viento hay, y cómo va a estar el clima las próximas horas y los próximos días.

🧸 **Imagínalo así:** es como tener un amiguito que sabe el clima de todo el planeta. Tú le gritas un nombre de ciudad y él corre, lo averigua, y vuelve con un dibujito y los números.

2. ¿De qué está hecha? Las piezas 🧱

El proyecto es como una **casita hecha de piezas**. Cada archivo hace una sola cosa:

Archivo	¿Qué es?
<code>main.jsx</code>	La puerta de entrada . Le dice al navegador: "pon la app dentro del div <code>#root</code> ".
<code>App.jsx</code>	El cerebro 🧠. Guarda toda la información y decide qué se ve.
<code>components/</code>	Las piezas de LEGO : la barra de búsqueda, la tarjeta de temperatura, la lista de días, etc.
<code>weatherIcons.js</code>	El armario de dibujitos : sabe qué dibujo va con cada clima.
<code>App.css</code> y <code>index.css</code>	La ropa 👕: colores, tamaños, dónde va cada cosa.
<code>.env</code>	El bolsillo secreto 🗝️ donde guardamos la llave de la API (nadie la debe ver).

3. ¿Qué es React? 🧩

React es una forma de construir páginas web con **bloques que se arman como LEGO**. Cada bloque se llama **componente** y es una función de JavaScript que devuelve un pedacito de página.

Nuestra app tiene estos bloques:

- **SearchBar** → la barra de arriba (búsqueda + ciudad actual + fecha)
- **WeatherCard** → la temperatura grandota de la izquierda
- **DailyForecast** → la lista de los próximos días (derecha)

- **HourlyForecast** → la franja de horas con la línea ondulada (abajo)
- **Loading** → el circulito que gira mientras esperamos
- **Footer** → el pie de página

💡 **La regla de oro: App es el papá** y los componentes son **los hijos**. El papá guarda todos los juguetes (los datos) y se los presta a los hijos. Los hijos nunca agarran nada solos: si quieren algo, le avisan al papá. Eso se llama **props** (los juguetes prestados) y **"lifting state up"** (los datos viven arriba, en el papá).

4. Los estados: cajitas con memoria

React tiene una magia llamada `useState`. Es una **cajita que recuerda algo**, y cuando cambias lo que hay dentro... ¡React vuelve a dibujar la pantalla solito!

```
// "weather" es la cajita, "setWeather" es la única mano que puede meter cosas
const [weather, setWeather] = useState(null)
```

El cerebro (App.jsx) tiene estas cajitas:

Cajita	¿Qué recuerda?
<code>city</code>	Lo que vas escribiendo en la búsqueda, letra por letra.
<code>weather</code>	El clima de AHORA (temperatura, viento, humedad...).
<code>forecast</code>	El pronóstico de 5 días (¡40 pedacitos, uno cada 3 horas!).
<code>loading</code>	¿Estamos esperando la respuesta? (sí/no) → muestra el circulito.
<code>error</code>	El mensaje de "ups, algo salió mal" si la ciudad no existe.
<code>suggestions</code>	Las ciudades que te sugiere mientras escribes.
<code>recentCities</code>	Las últimas 8 ciudades que buscaste.

 **Imagínalo así:** las cajitas son como pizarras mágicas. Si alguien escribe algo nuevo en una pizarra, toda la habitación se reordena sola para mostrar lo nuevo. Tú nunca mueves los muebles a mano: solo escribes en la pizarra.

5. ¿De dónde salen los datos? La API

Los datos vienen de **OpenWeatherMap**, una página que sabe el clima de todo el mundo. Una **API** es como un **teléfono para computadoras**: nuestra app "llama" a OpenWeatherMap y le pregunta. Para llamar necesitamos una **API key**: una **llave secreta** que dice "soy yo, tengo permiso". La guardamos en el archivo `.env` para que no se suba a internet con el código:

```
VITE_WEATHER_KEY=tu_llave_secreta
```

Y en el código la leemos así (Vite solo deja pasar las variables que empiezan con `VITE_`):

```
const API_KEY = import.meta.env.VITE_WEATHER_KEY
```

Hacemos DOS llamadas a la vez 📞📞

Cuando buscas una ciudad, la app hace dos preguntas **al mismo tiempo** con `Promise.all`:

1. `/weather` → "¿cómo está el clima AHORA?"
2. `/forecast` → "¿cómo va a estar los próximos 5 días?"

```
const [weatherRes, forecastRes] = await Promise.all([
  fetch(`${base}/weather?${params}`),
  fetch(`${base}/forecast?${params}`),
])
```

🐻 **Imagínalo así:** en vez de mandar a un amiguito a preguntar una cosa, esperar a que vuelva, y mandarlo otra vez... mandamos a **dos amiguitos a la vez**. ¡Así terminamos el doble de rápido!

¿Y si algo sale mal? 🚨

Un detalle importante: `fetch` **no se queja solo** si la ciudad no existe. Hay que revisar la respuesta a mano con `response.ok`:

- Error **404** → "Ciudad no encontrada" (escribiste mal el nombre)
- Error **401** → "Llave inválida" (la API key está mal o todavía no funciona)
- Cualquier otro → "Hubo un problema, intenta más tarde"

El mensaje se guarda en la cajita `error` y aparece en pantalla en un cartelito rojo. Y pase lo que pase, el bloque `finally` apaga el circulito de "cargando".

6. useEffect: "haz esto en el momento justo"

`useEffect` es como ponerle una **alarma** a React: "cuando pase tal cosa, haz esto". Lo usamos dos veces:

a) Cargar Buenos Aires al abrir la app

```
useEffect(() => {  
  fetchWeather(DEFAULT_CITY)  
}, []) // [] vacío = "solo UNA vez, al abrir"
```

El `[]` vacío al final significa: "haz esto **una sola vez**, justo cuando la app aparece". Por eso al abrir la página ya ves el clima de Buenos Aires sin buscar nada.

b) Las sugerencias mientras escribes (el "debounce")

Cuando escribes "bar...", la app le pregunta a la API "¿qué ciudades empiezan así?" y te muestra un menú (Barcelona, Bari...). Pero hay un truco: **no pregunta con cada letra**. Espera **350 milisegundos** a que dejes de escribir. Eso se llama **debounce**.

 **Imagínalo así:** si le preguntas algo a tu mamá letra por letra ("¿b?", "¿ba?", "¿bar?"), la vuelves loca. Mejor esperas a terminar la palabra y preguntas UNA vez. El `setTimeout` es la pausa, y si escribes otra letra antes de tiempo, `clearTimeout` cancela la pregunta vieja.

7. localStorage: el cajón que no olvida

Las últimas 8 ciudades que buscaste se guardan en `localStorage`, un **cajón del navegador** que recuerda cosas **aunque cierres la página o apagues la compu**.

- Cada vez que buscas, la ciudad nueva se pone **primera**, se quitan las repetidas, y solo se guardan 8.
- Al abrir la app, se lee el cajón **una sola vez** (eso se llama "lazy init") y la lista aparece en el menú de la flechita .

8. Las piezas, una por una

SearchBar — la barra de arriba

Tiene tres zonas: el logo a la izquierda, la **"píldora"** con la ciudad actual y la fecha en el centro, y la búsqueda a la derecha.

- La flechita  abre el menú de **ciudades recientes**.
- Al escribir aparece el menú de **sugerencias**; si tocas una, busca directo (con el país incluido, tipo "Roma,IT", para no confundirse con otra Roma).
- El input no cambia nada solo: **le avisa al papá App**, y App decide.

WeatherCard — la temperatura grande

- `Math.round(23.68)` → **24** (redondea, porque 23.68° se ve feo).
- La API da el viento en metros por segundo; lo multiplicamos **× 3.6** para tenerlo en km/h.
- La descripción viene ya en italiano porque la URL lleva `lang=it`.

DailyForecast — los próximos días

La API gratis **no** da "un resumen por día": da **40 pedacitos** (uno cada 3 horas). Este componente hace de **clasificador**:

1. Mete cada pedacito en la bolsita de su día (mirando la fecha "2026-07-10...").
2. De cada bolsita saca la temperatura **mínima**, la **máxima**, y el dibujito del **mediodía** (que es el más representativo del día).

 **Imagínalo así:** tienes 40 caramelos revueltos y los separas por color en 5 bolsitas. Luego a cada bolsita le pones una etiqueta con lo más importante.

HourlyForecast — la línea ondulada de abajo

¡La línea ondulada **no es un adorno**! Es un **gráfico de verdad**: sube donde hace más calor y baja donde hace más frío. Cada número está sentado sobre su punto de la curva.

- Se muestran **8 horas por página**; las flechas pasan de página (como un libro).
- Cada temperatura se convierte en un punto (x = su columna, y = qué tan caliente es comparada con las demás).
- Los puntos se unen con **curvas de Bézier**: el truco clásico para que una línea de picos se vea suavcita.
- El puntito naranja  marca la hora más cercana a AHORA.

⌚ Loading y 📄 Footer

- **Loading:** el circulito que gira. App lo muestra solo cuando `loading` es verdadero: `{{loading && }}`. Eso se llama **rendering condicional**: "muestra esto SOLO SI...".
- **Footer:** da las gracias a OpenWeatherMap y calcula el año solo con `getFullYear()`, para no tener que cambiarlo nunca a mano.

🎨 weatherIcons.js — el armario de dibujitos

La API dice el clima con un código ("01d" = sol de día, "11n" = tormenta de noche...). Este archivo agarra ese código y devuelve el dibujo correcto:

- **Iconos 3D grandes** (Fluent Emoji de Microsoft) → para la ilustración del centro.
- **Iconos de línea finita** (Meteocons) → para la lista de días.
- Si llega un código raro que no tenemos, usa el de "nublado" para no romperse.

9. ¿Y el CSS? La ropa de la app

- Toda la app vive dentro de **una gran tarjeta oscura redondeada** (la `.app-card`), sobre un fondo un poquito más claro: así la tarjeta parece "hundida" y elegante.
- El color de acento es el **naranja #ffb02e**, usado con cuentagotas: el puntito de la hora actual y la barrita del día de hoy.
- La nube 3D **flota** subiendo y bajando (animación `float`) y tiene un **resplandor cálido** detrás.
- El circulito de cargando es CSS puro: un aro gris con un ladito naranja que gira sin parar.
- Es **responsive**: en tablet el panel de días baja, y en celular todo se pone en una columna.

10. El viaje completo de una búsqueda

Cuando escribes "Tokyo" y aprietas Enter, pasa todo esto en orden:

1. El formulario dice "¡me enviaron!" y `preventDefault()` evita que la página se recargue.
2. Se enciende la cajita `loading` → aparece el circulito girando.
3. Salen los **dos amiguitos** a preguntar (clima de ahora + pronóstico).
4. Si la ciudad no existe → cartelito rojo de error. Fin.
5. Si todo va bien → se llenan las cajitas `weather` y `forecast`.
6. "Tokyo" se guarda primera en las recientes (y en el cajón `localStorage`).
7. Se apaga `loading` ... y React **redibuja todo solo**: temperatura, nube 3D, días y la línea ondulada. ✨

11. Mini-diccionario

Palabra	En cristiano
Componente	Una pieza de LEGO de la página.
Props	Los juguetes que el papá (App) le presta a los hijos.
Estado (useState)	Una cajita con memoria; si cambia, React redibuja.
useEffect	Una alarma: "haz esto en tal momento".
API	Un teléfono para que las computadoras se pregunten cosas.
API key	La llave secreta que te deja usar ese teléfono.
fetch	Hacer la llamada telefónica.
Promise.all	Mandar varios amiguitos a preguntar a la vez.
Debounce	Esperar a que termines de escribir antes de preguntar.
localStorage	Un cajón del navegador que no olvida nada.
Rendering condicional	"Muestra esto SOLO SI pasa tal cosa".
Lifting state up	Guardar los datos en el papá, no en los hijos.
Vite	El cocinero  que junta todos los archivos y sirve la app calentita en <code>npm run dev</code> .